

The 8086 Microprocessor

Prepared by:
Bipasha Majumder
Dept. of CSE, DIU

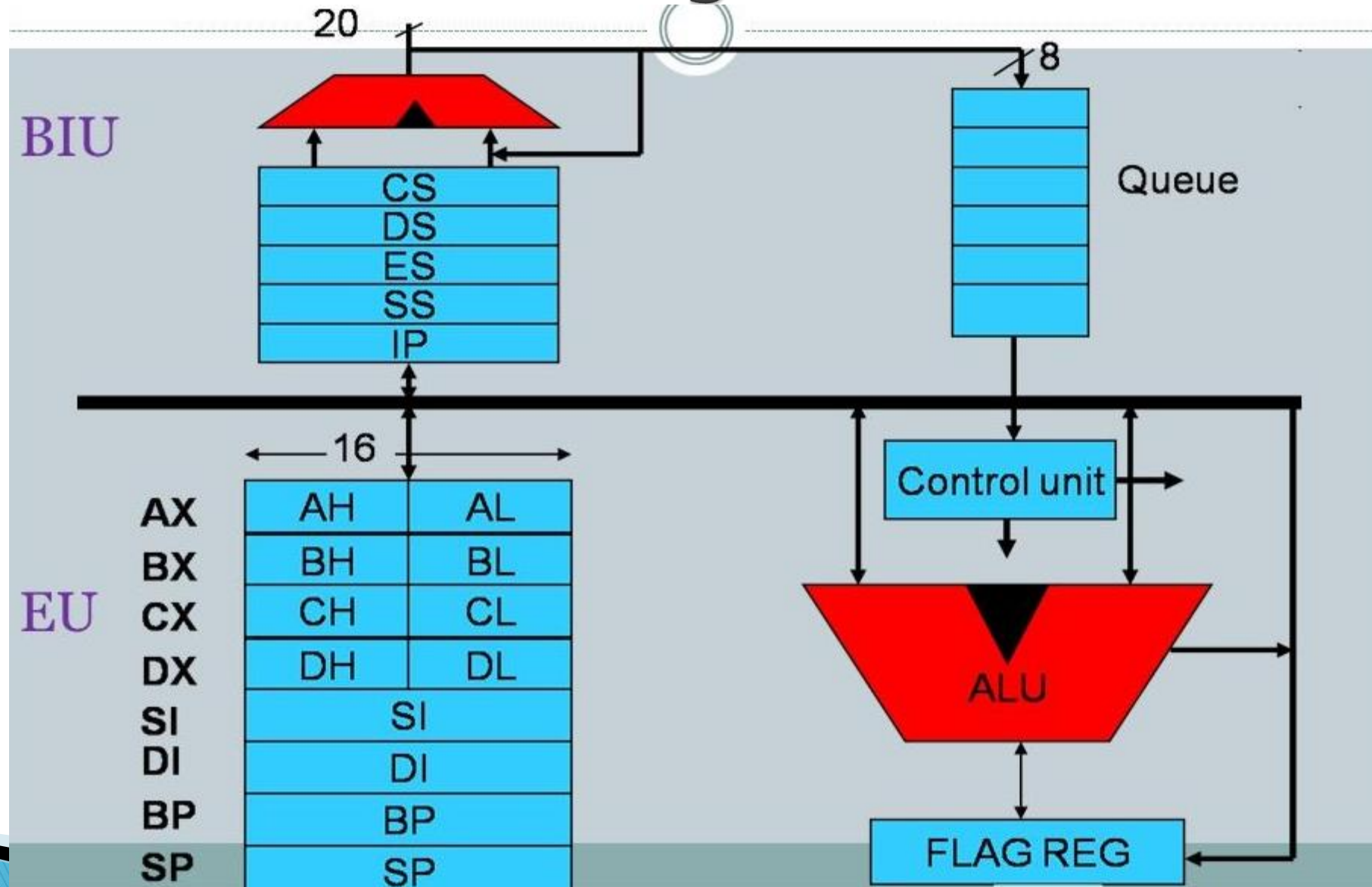
The 8086

- ▶ It is a 16-bit microprocessor
 - ▶ Its ALU, internal registers and most of the instructions are designed to work with 16-bit binary words
 - ▶ It has 16-bit data bus
 - ▶ It has 20-bit address bus
- 

8086 Internal Architecture

- ▶ 8086 cpu is divided into two independent functional parts.
 - ▶ Bus interface unit (BIU)
 - ▶ Execution unit (EU)
- 

8086 internal block diagram



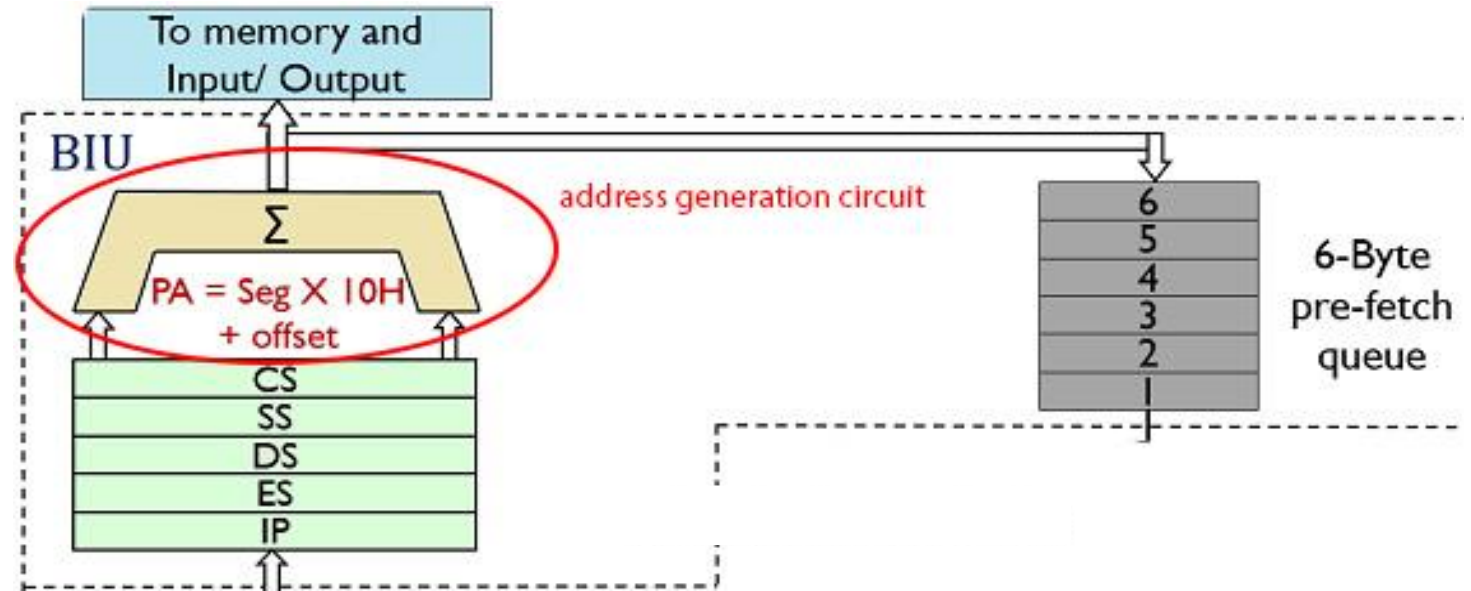
Bus Interface Unit – BIU

1. Instruction Queue–

- 6 byte queue
- Prefetch instructions(FIFO)

2. Segment Register–

Start address of a data. To get the exact location of data or instruction within a segment an offset value is required. To reference any memory location in a segment the processor combines the segment address in the segment register with the offset value of the location.



It has 4 segment registers that hold address of instruction and data in memory, which are used by the CPU to access memory location.

- ▶ Code Segment register
- ▶ Data Segment register
- ▶ Extra Segment register
- ▶ Stack Segment register

3. Instruction Pointer –

It's a 16 bit register that holds offset of the next instructions in the CS.

BIU perform–

- ▶ Instruction fetching.
 - ▶ Instruction queuing.
 - ▶ Communicate with i/o.
- 

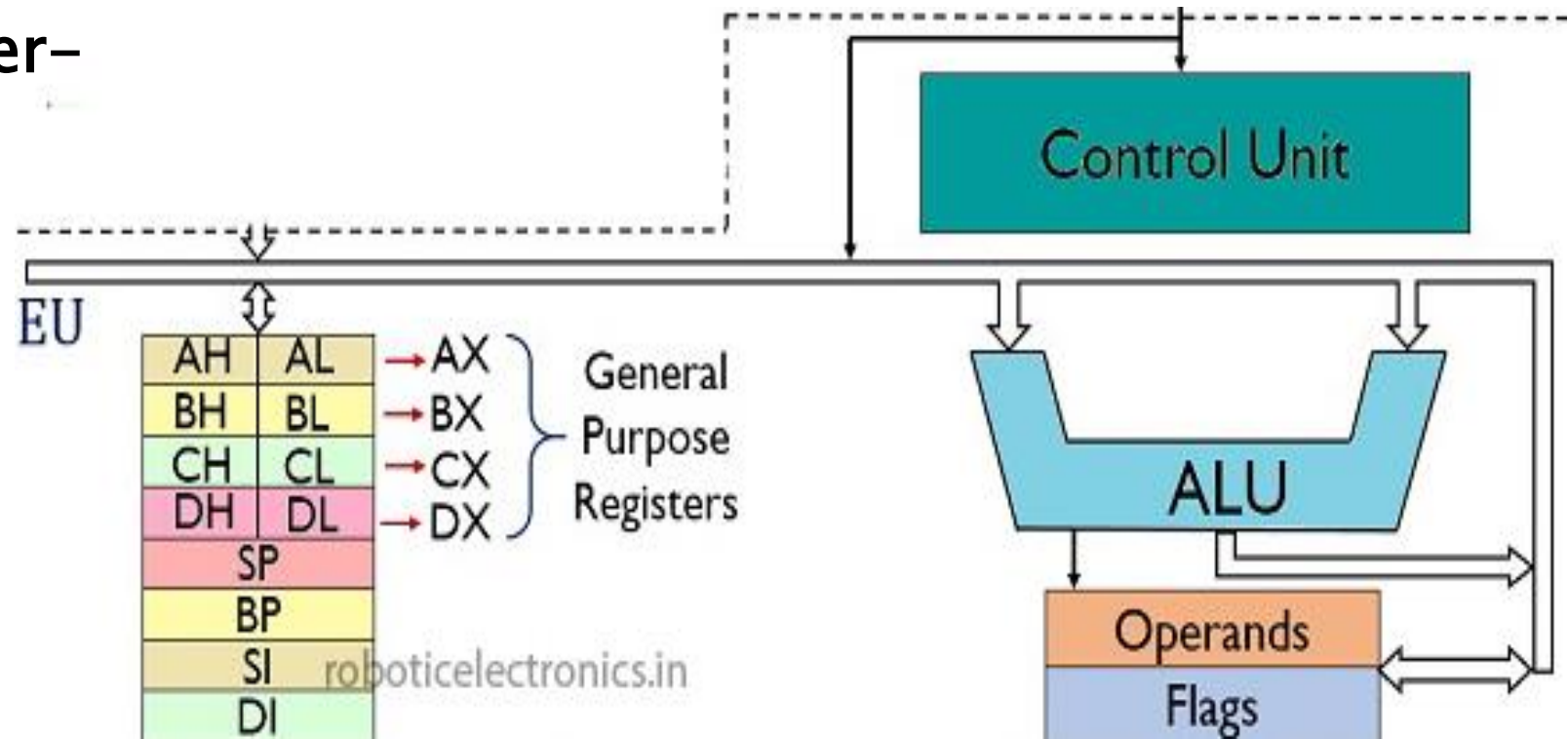
Execution Unit in 8086 (EU)

This is an execution unit where instruction decoding and execution takes place.

1. General Purpose Register–

Four 16 bit register divided into 2 parts low [8bit] and high [8bit] use for data handling.

- ▶ **AX:** This is known as the accumulator register. Store final result, handle i/o instruction, keeps updating.



- ▶ **BX:** This is called a Base register. It holds memory address in addressing mood.
 - ▶ **CX:** This is known as the Counter register. It holds count for instructions like loop/Shift/rotate and string operation.
 - ▶ **DX:** This is known as the Data register . Use for data multiplication used with Ax to hold 32 bit values during multiplication.
- 

2.Pointers register – contain offset address.

➤ SP(Stack pointer)– point to stack, top of stack segment.

$$PA = [SS] * 10H + [SP]$$

➤ BP(Base pointer)– use to access random location of stack.

$$PA = [DS] * 10H + [BP]$$

3.Index register –

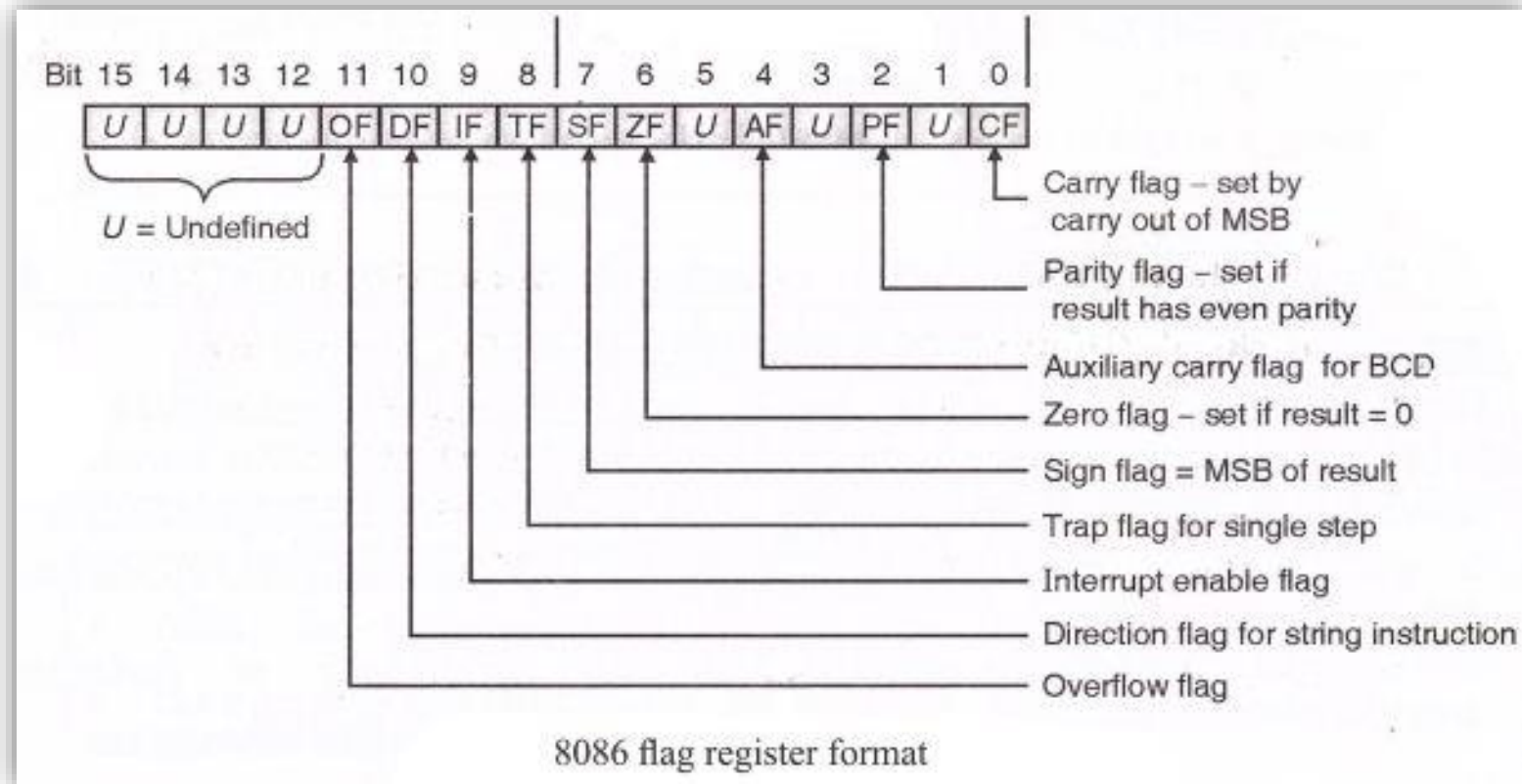
➤ DI(Destination Index)– holds offset address in extra segment during string operation.

➤ SI(Source Index)– holds offset address in data segment during string operation

4.Temporary register– store temporary

5. ALU– perform arithmetic and logical operation.

7.Flag registers



- ▶ The flag register is a 16-bit register in the Intel 8086 microprocessor that contains information about the state of the processor after executing an instruction. There are two types of flag register 1) conditional and 2) control
 - ▶ **Conditional flag:** it is used when ALU operation is performed.
 - ▶ **Carry flag**– It is set whenever there is a carry or borrow out of the MSB (most significant bit) of a result. D7 bit for an 8 bit operation and D15 bit for a 16 bit operation.
 - ▶ **Auxiliary carry flag**– this flag is set when ALU perform BCD operation of 8-bit and if result in a carry or borrow.
 - ▶ **Parity flag**– It is set if the result has even parity. If parity is odd, PF is reset. This flag is normally used for data transmission errors.
 - ▶ **Zero flag**– this flag is set 1 when the result of ALU operation is zero else it is set to zero.
- 

- ▶ **Sign flag**– It is set if the MSB of the result is 1. For signed operations such a number is treated as negative.
- ▶ **Overflow flag**– It will be set if the result of a signed operation is too large to fit in the number of bits available to represent it. It can be checked using the instruction INTO (Interrupt on Overflow).
- ▶ **Control flag**: it used to controls the operation.
- ▶ **Trap flag**– It is used to set the trace mode i.e. start single stepping mode. Here the microprocessor is interrupted after every instruction so that the program can be debugged.
- ▶ **Interrupt flag**– this flag is used to enable or disable interrupt. It is set to 1 for interrupt enabled and set to 0 for interrupt disabled
- ▶ **Direction flag**– It is used in string operation. As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-a-versa

- ▶ If the result of an 8-bit addition is 100011110, explain which flags are set and why?
- ▶ A signed 8-bit addition results in $(+127) + (+1) = ?$. Which flag(s) will be set, and why?